

Vediamo ora quanto impatto ha l'utilizzo delle periferiche esterne sul nostro programma (ricordiamo che incremento delle prestazioni sono del 60%, mentre il ritardo dei componenti fisici si aggira intorno al 10%): vediamo la **legge di Amdal**: quanto inserendo un processore più veloce, permettere di avere un incremento delle prestazioni:

$$S = \frac{1}{(1-f) + \frac{f}{K}}$$

Dove f è la parte di attività passata in processamento dalla cpu, 1-f quella dell' interazione con dispositivi, mentre k è il fattore di volte che voglio il processore più veloce.

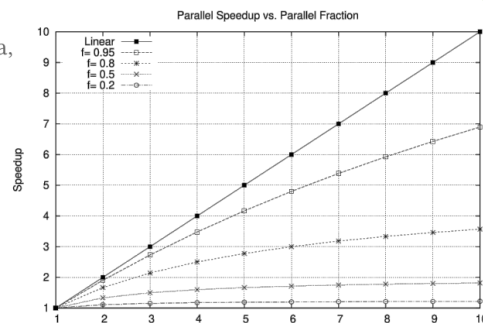
Vediamo un esempio:

I dispositivi di I/O determinano un collo di bottiglia prestazionale

Anche se la parte f è elevata, lo speedup sarà limitato dall'I/O

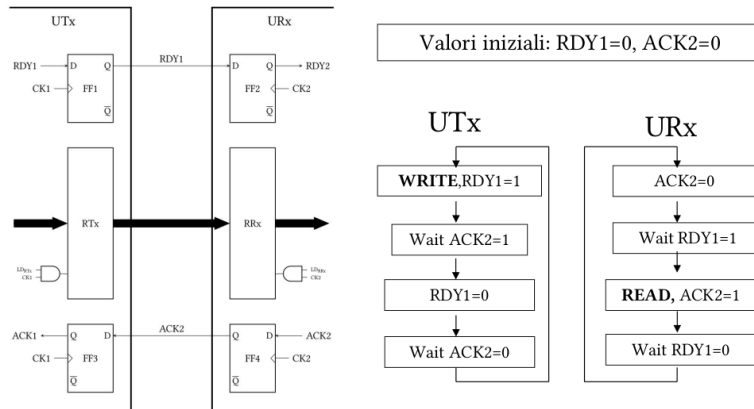
Ad esempio, se $f = 80\%$:

$$\lim_{K \rightarrow \infty} S = \frac{1}{1 - 0.80} = 5$$

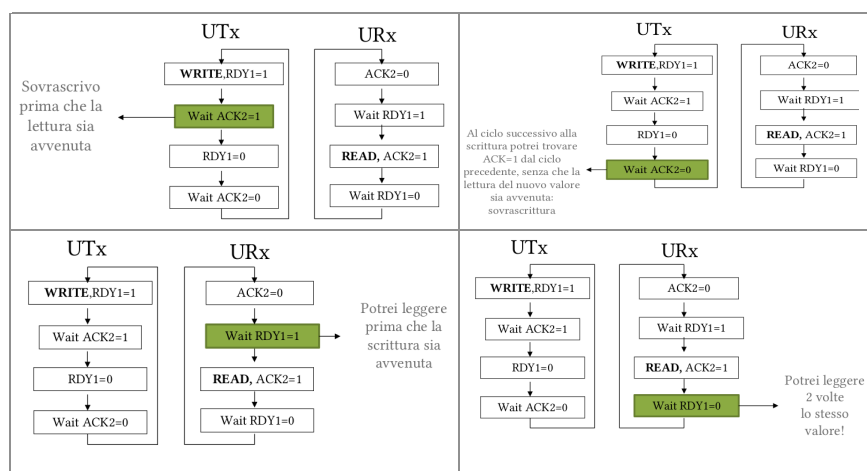


Come facciamo però in realtà a fare comunicare 2 o più dispositivi? Ci deve essere sia un canale di comunicazione, che la stessa velocità: quindi in questo caso si parla **PROTOCOLLO DI HANDSHAKING**: innanzitutto si "presentano" i dispositivi, poi c'è lo scambio di informazioni per far viaggiare i dati ad una stessa velocità (stesso clock): quindi si deve cercare di mantenere dati in modo stabile. Quindi viste le differenze di velocità e la necessità di mantenerli stabili, vi si devono usare dei registri tampone.

Vediamo il circuito:

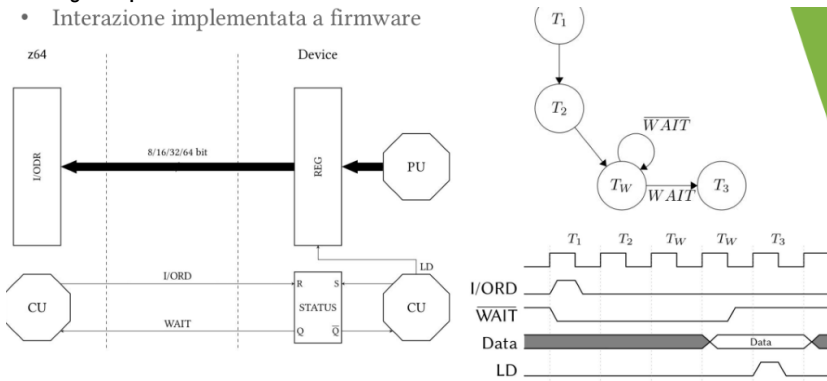


Focalizziamoci ora sui FF: ci basta un ulteriore bit per sapere se c'è un dato da leggere, mente il secondo bit serve a dire "ho finito di leggere". **Anche questi due segnali devono essere bufferizzati**. Quindi usando questi due registri tampone permettono inoltre la comunicazione tra registri e FF, in quanto hanno velocità lettura e scrittura differenti. Quindi in dettaglio:

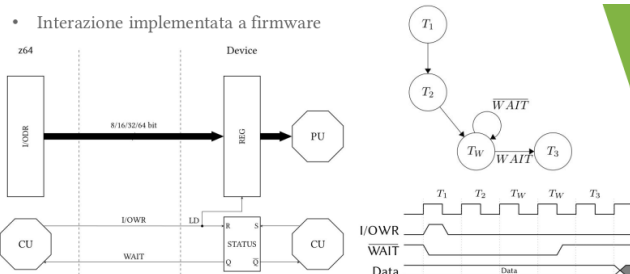


In dettaglio : input

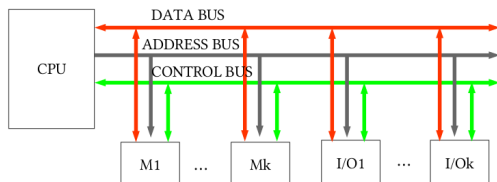
- Interazione implementata a firmware



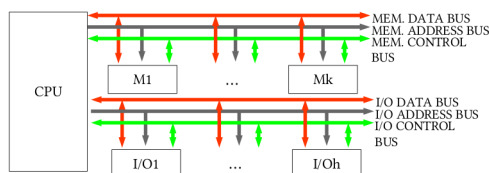
Innanzitutto ricordiamoci che il processore è molto più veloce dei dispositivi, quindi dovrà attendere che producano dati, mettendosi in attesa. Quindi si ha un solo FF (chiamato stato) che utilizza il processore per chiedere al dispositivo di mandare i dati. Quindi l'unità di controllo (CU) invia il segnale io/rd (lettura da input/output), il quale segnale viene scritto nel ff contenuto nel processore. Dato che il dato è cambiato, ed il processore quindi ha resettato il ff (segnale reset), inizia a produrre i dati, i quali verranno mandati sul registro di interfaccia. Dati pronti e scritti quando la CU abilita il segnale di io/rd. Attenzione anche al segnale di wait (aspetta): vale 0 fin tanto che il processore deve aspettare finché l'UC (unità di controllo) scrive i dati sul registro, vale 1 (settato) quando i dati sono stati scritti. Se nel ff si ha il set (segnale ad 1) anche il wait cambia: si ha quindi transizione di stato. **Andiamo ora a vedere l'output:**



Cambia il modo in cui andiamo a scrivere i dati nel registro. Il processore bufferizza i dati nel registro di interfaccia, manda il segnale di io/wrt al ff. Attenzione al collegamento del segnale LD (dove è). Per il resto è analogo. **Questo schema è quello che si usa, ma dal punto vista prestazionale fa schiantare il processore: spreca infinità di ciclo di clock.** Andiamo a vedere come generalizzare questa connessione: si hanno dei bus di interconnessione, i quali viaggiano all'interno della scheda madre, per andare a connettere i vari pezzi:

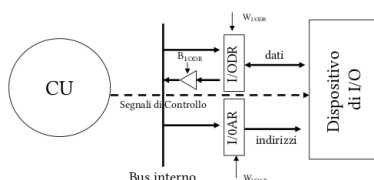


Notiamo che in questa configurazione, c'è un solo bus condiviso con la memoria (bus di sistema), ed è efficiente perché riduce il numero di fili, ma vediamo che il bus di indirizzi viene usato dalla memoria e dai dispositivi, non si sa con chi parlo: sia hanno spazi di indirizzamento condivisi (dispositivi rubano indirizzi alla memoria): configurazione **memory mapped**. Vedasi istruzione mov. In alternativa: utilizzo altro bus: un bus per la memoria ed un altro dedicato per i dispositivi i/o:

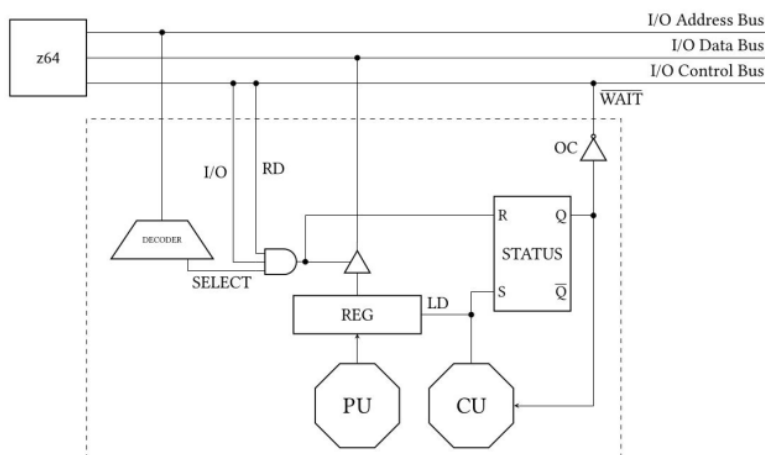


Ha spazi indirizzamento diversi: possibile utilizzo tutti e 64 bit, ma avendo due bus diversi, ho dei registri di interfaccia per ognuno di questi bus. Vedasi registri MAR e MDR. Quindi per quanto detto ora dobbiamo modificare l'interfaccia dello z-64, per consentire di operare su più bus: quindi si parte da questa:

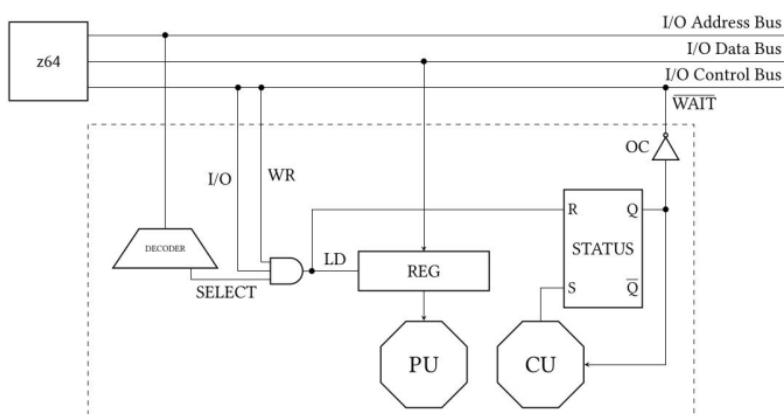
- Registro Dati (I/ODR)
- Registro Indirizzo (I/OAR)
- Segnali di Controllo (I/O, RD, WR, ...)



e si arriva a questa (per input) :



In dettaglio : andiamo a vedere il decoder : da un ingresso di 64 bit, attiva una sola uscita , avendo così un solo dispositivo attivo. Quindi dopo questa fase i segnali i/o e rd arrivano a tutti i dispositivi , ma avendo la combinazione in and con indirizzo di un solo dispositivo . poi il processore abilita il buffer tri state per scrivere i dati sulla data bus ed evitare che il ff si bruci (ingresso q). Per quanto riguarda anche il wait , anche qui altro buffer tri state per evitare che il segnale q non si propaghi ad altri ff, usando la tecnologia di Open-collector : uso il not per calcolare l'or cablato di tutti i dati che arrivano dai dispositivi di input : il processore si mette in wait finché almeno un dispositivo che produce dati. **Nel caso dell'output :**



Il processore mette dati sul data bus: unico dispositivo determinato dal decoder , permettendo così il flusso dei dati ed il dispositivo può incominciare a processarli. Analogamente tecnologia OC del ff . Quindi per interazione multi-dispositivo si usa una linea di abilitazione alla quale risponde uno ed uno solo dispositivo alla volta. Andiamo ora a vedere i tipi di interazione tra cpu e dispositivi : cominciamo con **I/O programmato** : nel quale si ha che è il programma sw che inizializza e governa le interazioni con i dispositivi per effettuare il trasferimento dei dati , poi vediamo **richiesta esterna** : il dispositivo richiede attenzione della cpu , la quale esegue opportuno driver per gestire la richiesta ed il trasferimento dei dati, infine invece si ha : **interazioni gestite da processori dedicati** : il processore trasferisce la responsabilità della richiesta e/o gestione ad un coprocessore. **Dedichiamoci in dettaglio all'i/o programmato** : in questa interazione è il software che guida lo scambio di dati tra periferiche e cpu , quindi il software necessita di particolari istruzioni per poter effettuarla , le quali sono :

Instruction	Syntax	Semantics
Inbound transfer from parametric I/O port	inX %dx , RAX	Transfer data of size X from the device deployed on the I/O address contained in the %dx register.
Inbound transfer from explicit I/O port	inX \$ioport, RAX	Transfer data of size X from the device deployed on the I/O address \$ioport.
Outbound transfer to parametric I/O	outX RAX, %dx	Transfer data of size X to the device deployed on the I/O address contained in the %dx register.
Outbound transfer to explicit I/O	outX RAX, \$ioport	Transfer data of size X to the device deployed on the I/O address \$ioport.

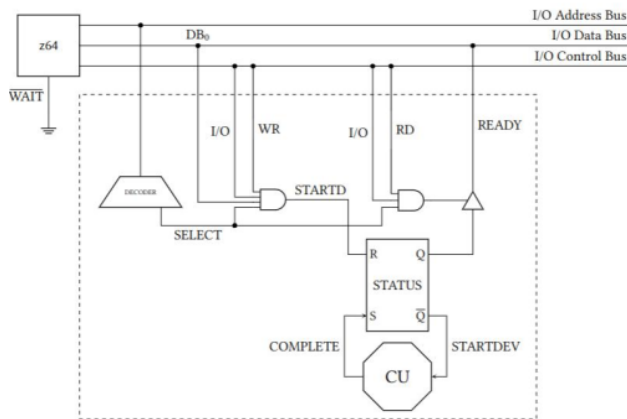
Dove %dx contiene fino ad un massimo di 2^{16} porte : ovvero questo registro contiene al massimo 2^{16} possibili indirizzi. Questo tipo di interazione avviene secondo due tecniche : **busy waiting e polling** . Per quanto riguarda il primo si ha che avviene attraverso 4 passi :

1. Il processor avvisa il dispositivo che vuole effettuare un trasferimento , secondo il modello vista prima.
2. Il preprocessore verifica il bit di status (bit del ff) se è resettato(0) oppure settato(1)
3. Se il ff (status) è resettato , il preprocessore continua ad aspettare , mettendosi nello stato di wait
4. Il programma continua il suo normale flusso

Quindi in questa modalità non è possibile esplicitamente interazione tra bus e dispositivi , almeno che non è la cpu ad interrogarli direttamente. quindi non vi è più del famoso segnale di wait negato : viene quindi collegato a massa . Sembra efficiente questa modalità ma in realtà soffre delle stesse

problematiche dell' i/o implementato a firmware , ed in particolare il processore non esegue nessun'altra attività fino al completamento del trasferimento dei dati.

Quindi andiamo a rivisitare l'architettura per i/o :



Da notare in questa architettura : **innanzitutto il segnale di wait negato viene messo a massa, levandolo così la tecnologia open-collector** (porte and per ottenere or logico) , poi in base and indirizzo sul bus e decoder , viene selezionato un solo device , il quale device prende il dati dal data bus, ed abilitando i segnali di i/o e wr (scrittura su device) , va a resettare il ff (status) che essendo 0 , il processore aspetta finché non viene settato(1). Per quanto invece riguarda la lettura il procedimento è uguale : si parte da indirizzo-> decoder->1 solo dispositivo, poi abilitando il segnale i/o e lettura , il dispositivo va a leggere i dati dal bus ed attiva o meno un buffer tri state per dire al processore che ha finito. Per quanto riguarda i segnali invece : **STARTD**: Questo segnale identifica la volontà del processore di avviare un protocollo di scambio dati con il device. Nello specifico questo segnale resetta il FLIP FLOP di status. **STARTDEV**: È il buffering del segnale STARTD. Lo scopo di questo segnale (che diventa un ingresso della CU del dispositivo) è quello di generare una transizione di stato affinché l'operazione di interesse (produzione o consumazione dati) possa essere avviata. **COMPLETE**: Questo è il segnale di controllo generato dalla CU del dispositivo. Questo segnale imposta il valore del FLIP FLOP STATUS ad 1, indicando che l'operazione richiesta dalla CPU è stata eseguita. In dettaglio (almeno in assembler e concettualmente) :

```
movb $1, %al
outb %al, $device
.bw:
inb $device, %al
btb $0, %al
jnc .bw
```

- Problema: il processore esegue lo stesso codice finché non è completato il trasferimento
- Nonostante il processore non sia in *stallo*, non vengono effettuate attività utili
- Possibile soluzione: interrogare altre periferiche e servire la prima disponibile

Andiamo ora a vedere la seconda tipologia : **il polling** : è simile al busy waiting , in quanto effettua una verifica circolare per vedere la prima periferica pronta ad interagire . Vediamolo in dettaglio :

```
.poll :
movw $STATUS_DEV1, %dx
inb $STATUS_DEV1, %al
btb $0, %al
jc .dev1
inb $STATUS_DEV2, %al
btb $0, %al
jc .dev2
# ...
jmp .poll

.devX:
# ...
jmp .poll
```

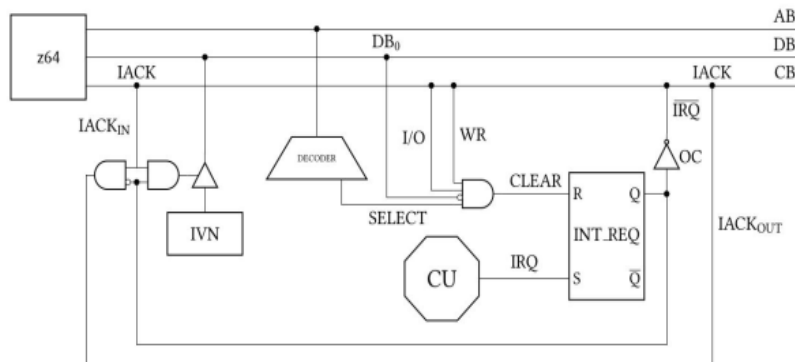
- Problemi: il processore deve continuamente testare il valore di STATUS
- Codice difficile da mantenere: cosa succede se si aggiunge un dispositivo?
- Rischio di *starvation*: le ultime periferiche possono non essere servite mai
- Difficile gestire le *priorità*

Andiamo ora a vedere come alterare il flusso del programma , magari per far si' che il flusso ne sia alterato : le **interruzioni**: meccanismo che forza il processore ad interrompere il normale flusso di esecuzione in favore di un altro flusso di esecuzione : quindi per fare si che ciò avvenga, vi deve essere eterogeneità tra i dispositivi interconnessi alla cpu. Quindi per fare si che ciò avvenga, i dispositivi alzano una "sorta di bandierina bianca" , e quando questa viene riabbassata(richiesta eseguita) ,il processore riprende la normale esecuzione. Questa gestione è priva di rischi?? Assolutamente no , in quanto la generazione di un'interruzione(alzo bandierina bianca) è *asincrona* , più dispositivi allo stesso istante richiedono attenzione cpu, il processore deve vedere quale dispositivo ha sollevato interruzione per poterlo gestire correttamente(driver) , ed infine la cpu può eseguire un solo driver alla volta . Quindi come possiamo gestirle in modo corretto? O ad ogni interruzione gli diamo una priorità (**daisy chain**) oppure realizzare un protocollo per identificare i dispositivi che hanno sollevato l'eccezione. Prima di addentrarci in queste gestioni , vediamo come si gestisce una richiesta di interruzione:

1. Si deve salvare lo stato attuale del programma (registri)
2. Identificare il driver per gestire in modo corretto il dispositivo che ha generato interruzione

3. Esecuzione del driver
4. Ripresa della normale esecuzione del programma

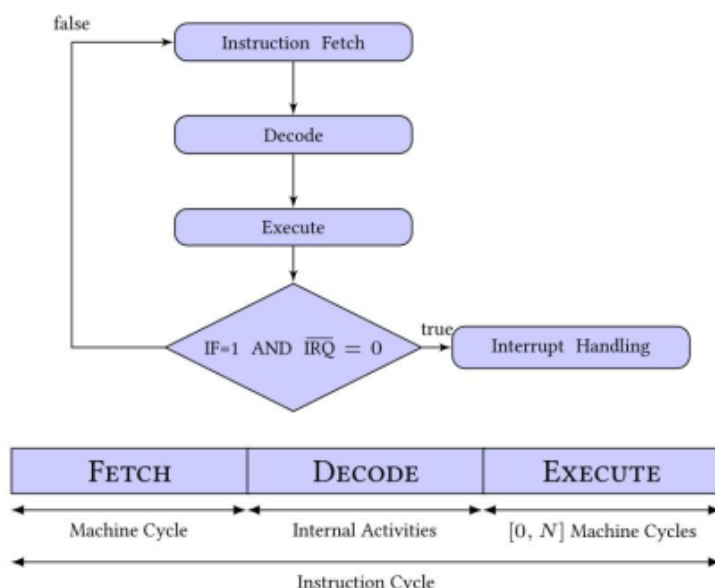
Andiamo a vedere il **daisy-chain**:



In questo schema il processore utilizza un segnale particolare (jack)->interrupt ack per dire che è pronto a servire una richiesta d' interruzione . Analogamente a prima , per scegliere il dispositivo da servire (con relativo driver) si va ad usare l'indirizzo preso e codificato dal decoder per scegliere il dispositivo (più vicino è ad indirizzo e più sarà servito rapidamente) , poi abilitando i segnali di i/o e WR si va a fare l'interazione. Per quanto riguarda il FF(stato interno) viene chiamato interrupt-request il quale viene posto a 0 (reset) se il dispositivo/processore ha eseguito il driver e questo segnale torna sul bus controllo attraverso logica di open-connector(and che si comportano da or) . Tornando invece al segnale di jack , prima di tornare sul bus di controllo , va in and con l'uscita del ff (interrupt-request) , e torna sul bus , ma se interrupt-request settato (1) il processore scrive altrimenti no(notare che q in entrata alla and è negato). Così funziona la gestione dell'interruzione, ma come fa il processore a scegliere quale driver usare per gestire?? In base all'**IVN (interrupt vector number)**: in base al numero si attiverà un'opportuna funzione di gestione(driver). Vediamo un semplice esempio (eseguito su un processore multi ciclo):

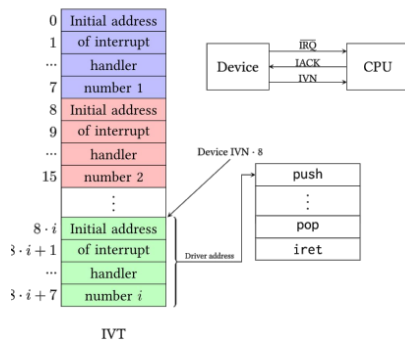
```
movq $0, %rax
testq %rax, %rax
```

Una richiesta di interruzione potrebbe essere ricevuta o durante il movq oppure tra movq e testq (test se registro è 0) , quindi in generale potrebbe succedere che : i registri invisibili al programmatore potrebbero essere corrotti , registro flags corrotto oppure il contenuto del registro %rax è stato corrotto. Quando da normale esecuzione del programma , si passa alla gestione del driver (basato su IVN) , si ha a che fare con quel meccanismo che prende il nome di **cambio di contesto** : prima salva lo stato del programma (registro RIP ->indirizzo da cui riprendere esecuzione, registro FLAGS bit di condizione , registri invisibili al programmatore Temp1/Temp2 , registri general purpose) su stack -> interrupt frame , poi attiva il programma per la gestione del driver. Attenzione però al costo : se architettura troppo complessa , il costo sale!! In generale nel processore ci deve essere un meccanismo , che impedisce al processore di seguire più gestioni di interruzioni contemporaneamente /anche la stessa (gestione attività atomica). Capita spesso che il processore deve "mascherare" l'ivn a 0 : quindi il processore va a modificare il *if (interrupt flag)* , attraverso lo sti/cli . Questo flag viene azzerato dal firmware durante il cambio di contesto. In virtù di quanto detto finora, vediamo ora come preservare i registri **soprattutto RIP e FLAGS**:



Il microprogramma viene eseguito completamente così' non si ha la corruzione dei registri invisibili al programmatore (registri salvati in modo consono). La gestione fatta finora delle interruzione sembrerebbe bastare , ma in realtà no in quanto il processore non sa quale è realmente il driver da eseguire : manca il collegamento tra IVN (numero che identifica il

dispositivo) e lo usa come indice di vettore di puntatori , e si spiazza di $8 \cdot \text{ivn}$ e IVT(interrupt vector table): tabella che contiene riferimenti a prime istruzioni per la gestione opportuna del driver:



Vediamo ora come avviene a livello istruzioni microcodice:

<pre> FLAGS[I] ← 0; TEMP1 ← RSP TEMP2 ← 8 RSP ← ALU_OUT[SUB] MAR ← RSP MDR ← RIP (MAR) ← MDR TEMP1 ← RSP TEMP2 ← 8 RSP ← ALU_OUT[SUB] MDR ← FLAGS MAR ← RSP (MAR) ← MDR IACK IACK; MDR ← IVN TEMP2 ← MDR MAR ← SHIFTER_OUT[SX, 3] MDR ← (MAR) RIP ← MDR </pre>	Setto if a 0-> niente interruzioni; mi salvo RSP in temp1 Decremento stack Rsp lo salvo nello stack Push del contenuto di flags Segnale di interruzione Ivn viene scritto sul bus Shifto a sx mdr di 3 posizioni (indirizzo da cui leggere indirizzo prossima istruzione) Prendo il valore dell'indirizzo iniziale del gestore driver Fetch prima istruzione
<pre> MAR ← RSP MDR ← (MAR) FLAGS ← MDR TEMP1 ← RSP TEMP2 ← 8 RSP ← ALU_OUT[ADD] MAR ← RSP MDR ← (MAR) RIP ← MDR TEMP1 ← RSP TEMP2 ← 8 RSP ← ALU_OUT[ADD] FLAGS[I] ← 1 </pre>	cima stack Leggo cima stack Sposto stack in flags Incremento stack di cella Mi sposto nella cella Sposto il contenuto in mar Nel bus ci metto il contenuto In rip ci metto indirizzo prossima istruzione Incremento stack Metto ad 1 if (interrupt flags)

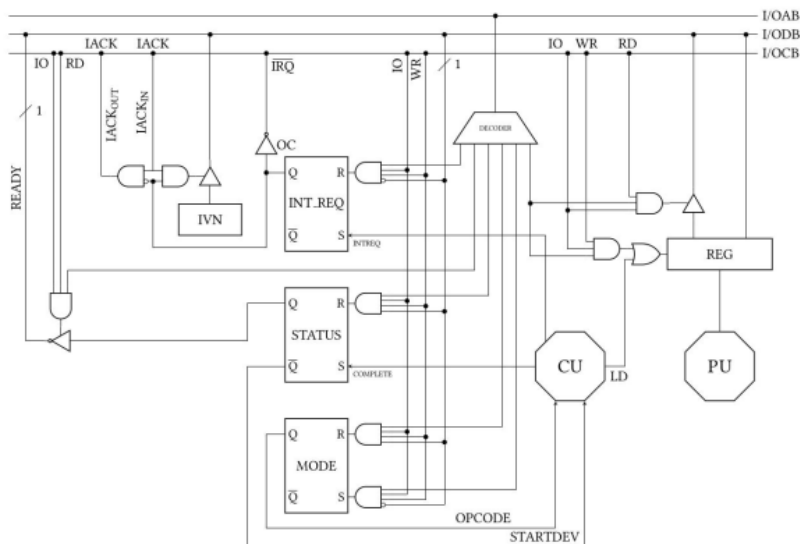
Ricordiamo che x0800 (.org 0x800) perché questa è la taglia dell'interrupt vector. Ricordiamo anche che : l'esecuzione di un qualunque driver avviene nella sua interezza prima di restituire il controllo (atomicità). I driver della periferica si dividono principalmente in due :

1. Top half : parte di lavoro non rinviabile eseguita con if=0
 - a. Recupero dei dati dal dispositivo
 - b. Se necessario riprogrammazione della periferica
2. Bottom half : parte rimandabile a priorità inferiore
 - a. Processamento dei dati
 - b. Può essere eseguita anche con le interruzioni abilitate
 - c. Sti (istruzione esplicita)

Vediamo un esempio :

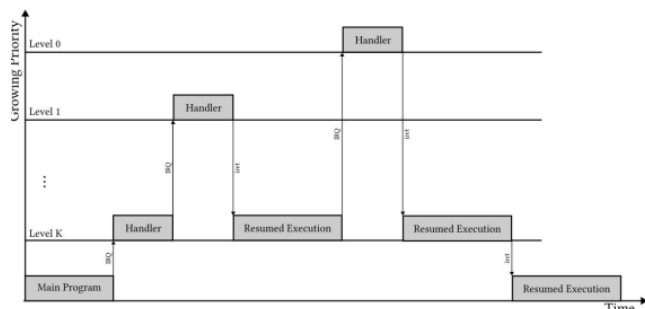
<pre> . driver 1 # save registers clobbered in the driver pushq %rax # read the data coming from the device # make a temporary copy in memory inw \$device_reg, %ax pushw %ax # delete the cause of the interrupt movb \$0, %al outb %al, \$device_irq # enable reception of interrupts : start of the bottom half sti # do any action to process the acquired data popw %ax </pre>	<pre> # restart the device to produce new data # (if the device is supposed to be used like this) movb \$1, %al outb %al, \$device_status # Destroy the interrupt frame and return control to the # interrupted program popq %rax iret </pre>
---	---

Attenzione : è possibile che un dispositivo operi sia in busy waiting che in asincrona , ma alla fine è il processore che sceglie quale modalità : arrivando così alla modalità mista :

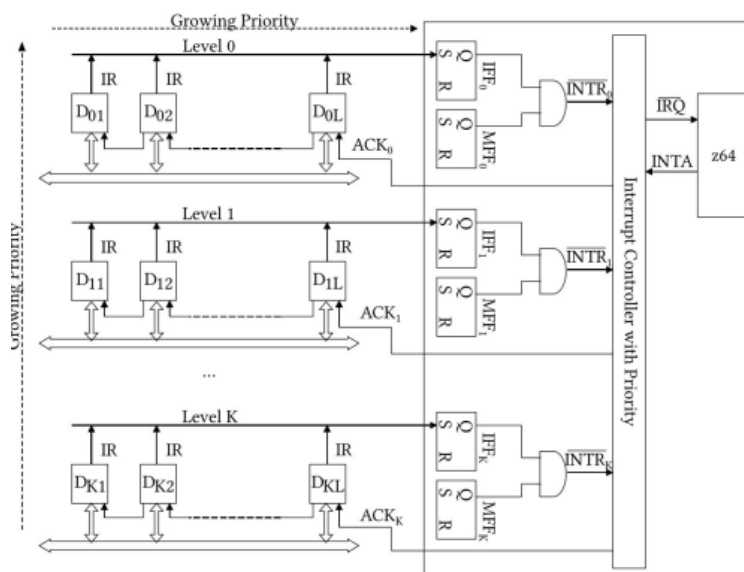


Focalizziamoci ora sulla priorità , andando a vedere i due più comuni problemi che possono sorgere :

Priority inversion e starvation. La prima consiste nel cambiare la priorità di un dispositivo : se un dispositivo effettua un'interruzione (quindi il processore attiva il driver) , almeno che nel driver vi sia istruzione esplicita si sti (cambia interruzione) , questo non può essere interrotto . Mentre la seconda si ha quando magari c'è un dispositivo a priorità alta e molto veloce , e siccome il processore serve solamente ed esclusivamente questo dispositivo , magari un dispositivo a priorità minore non verrà mai servito (neanche con schema daisy chain). Supponiamo di avere livelli di priorità : almeno con questa soluzione si può interrompere esecuzione driver priorità maggiore , eseguendo driver di quello priorità minore.



Semberebbe funzionare, ma attenzione al ciclo infinito causato dal bit di flag . Quindi si cerca un trade-off : organizzo i nostri dispositivi in determinati classi di priorità :



Per ogni dispositivi organizzati in daisy chain e per ogni classe di priorità vi è un solo segnale di interrupt request , in modo che il processore attivi il driver associato ad uno di questi dispositivi (stessa classe). In generale per qualunque richiesta di interruzione (qualunque classe), il processore non deve più mascherare il bit, ma può benissimo eseguire altre richieste di classi superiori, ignorando così le richieste multiple di uno stesso livello. **Quindi con questa soluzione si risolve l'inversione di priorità.** Per quanto riguarda invece la starvation un modo possibile è il seguente : è possibile dare più priorità a classe minore , usando/settando in modo opportuno i ff

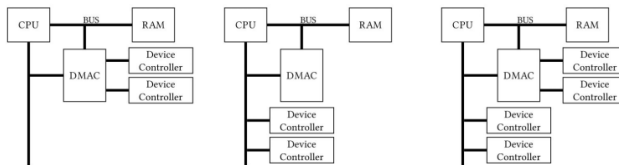
(memory mapped), mascherando così le richieste delle classi a livello inferiore. Quindi vista l'efficienza (non buona ma insufficiente), si opta per usare un canale dedicato, che nonostante i due cicli per trasferire data (1 per il fetch ed altro per trasferimento), si ha che comunque non basta: in caso di trasferimento di mole di dati molto grandi è lenta:



Accesso a memoria indiretto

Accesso a memoria diretto

Quindi si opta per una soluzione più efficiente: inserisco un coprocessore (controllore di bus) che effettua trasferimenti tra dispositivi e memoria a posto della CPU: si parla di **DMAC (Direct memory access control)**:

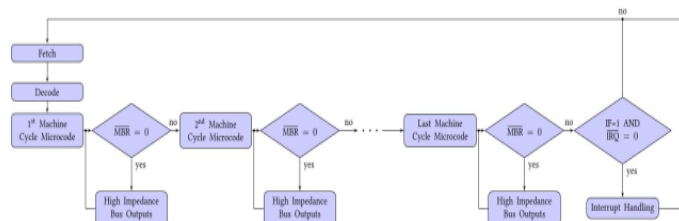


Periferiche invisibili alla CPU

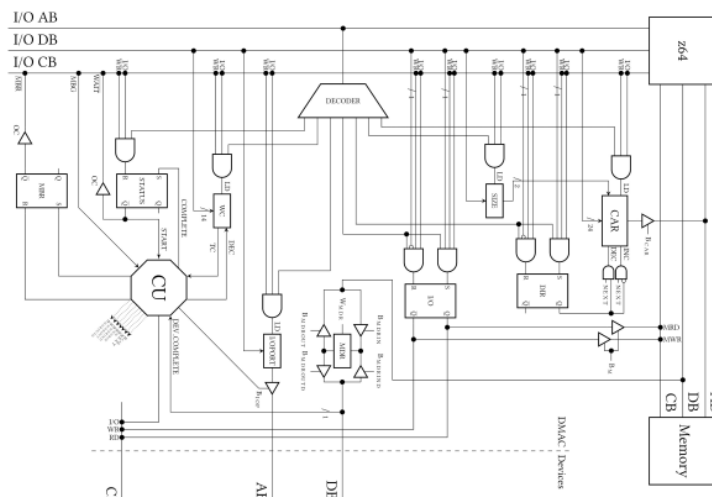
Periferiche condivise con la CPU

Modalità mista

Nel primo modo si ha un vantaggio quando ci sono dispositivi lenti, la seconda si ha quando le periferiche sono condivise tra processore e dmac, mentre nell'ultima quelli più lenti solo dal dmac, mentre per quelli più veloci o processore o dmac: questa scelta viene fatta dal sw. **Anche il dmac viene visto come dispositivo programmabile. Ma attenzione al bus condiviso: se entrambi inviano dati su stesso bus esplode tutto.** Quindi per evitare di fare ciò si deve progettare un meccanismo che permette di coordinare CPU e DMAC. Ma come si fa? Costruiamo un protocollo di handshaking tra loro: mutua esclusione dell'utilizzo del bus condiviso: usiamo due segnali di controllo: **MBR (memory bus request)**: utilizzato dal dmac per effettuare un trasferimento. Lavora in logica negata (tecnologia open collector): per evitare più richieste in parallelo (quindi più dmac lavorano in parallelo) ed il processore usa il **MBG (memory bus grant)**: segnale di alta impedenza che permette di staccarsi dal bus di sistema (isolamento): in sintesi con questo segnale si dice che il dmac ha controllo assoluto sul bus. Finché MBR negato vale 1 (asserito) il processore è in stallo. **Quindi in generale si deve minimizzare il tempo che il dmac invia dati (processore isolato).** Vediamo ora come minimizzare: due tecniche **burst mode e bus stealing**. Nella prima quando il processore asserisce MBG, il dmac mantiene MBR negato asserito, finché non ha completato il trasferimento dei dati: modalità inefficiente perché se grande quantità di dati si ha sottoutilizzo processore. Per migliorarla si passa alla seconda, la quale il dmac richiede il bus solo quando ha un dato da trasferire: dato a 32 bit immagazzinati nel registro tampone. Quindi effettuando questo trasferimento a pezzi di 32 bit, entrambi i dispositivi riescono a usare il bus condiviso.



Quindi il circuito è il seguente:



Notiamo che CAR (current address register) mantiene l'indirizzo corrente verso quale scrivere/leggere. Può essere scritto direttamente dal processore dello z64. Massimo 2^{24} bit

indirizzi (separazione degli indirizzi). Notiamo anche il registro size : specifica la grandezza dei byte da copiare (leggere/scrivere) : byte, word, long word, quad word a partire dal car. Quindi CAR è una sorta di registro contatore ! Il quale usa anche il df (direction flag) copiato all'interno del ff dell'interfaccia (DIR) (per sapere dove scrivere/leggere) . WC (word count) è un registro che dice quante parole di una determinata taglia dobbiamo trasferire (avanti/indietro) a partire da CAR. WC se arriva a 0 fornisce TC (terminal count) : trasferimento completato : analogo allo zf (zero flag)). Per quanto riguarda invece il dispositivo con cui dobbiamo interagire : prendo indirizzo e lo scrivo in porta i/o (registro) ; per scegliere invece il tipo di trasferimento lo sceglie il sw scrivendo il valore all'interno di registro di interfaccia (I/O) , il quale permette MRD e MWR (verso memoria) oppure sul control bus per interazione con dispositivi. Quindi riassumendo il tutto : il processore deve scrivere indirizzo da cui partire , la direzione in cui effettuare la scrittura, la taglia del singolo blocco , se è operazione di i/o, quante parole che devono essere scritte . Il ff di status mantiene tutte le info. Vediamo ora le istruzioni che permettono di fare ciò :

Instruction	Syntax	Semantics
Inbound transfer of a data string	insX	Transfer an arbitrarily large buffer of data from a device.
Outbound transfer of a data string	outsX	Transfer an arbitrarily large buffer of data to a device.

Sono istruzioni di tipo *stringa*

insX: leggi 10 byte da DEV

```
1 movq $10, %rcx
2 movq $dest, %rdi
3 movq $dev_mem, %dx
4 cld
5 insb
```

outsX: scrivi 10 byte su DEV

```
1 movq $10, %rcx
2 movq $dest, %rsi
3 movq $dev_mem, %dx
4 cld
5 outsb
```

L'esecuzione del trasferimento è *sincrona*: il DMAC asserisce $\overline{\text{WAIT}}$

Quindi : le architetture più moderne sono viste come **adattatori di bus (coprocessori che implementano interfacce per effettuare trasferimenti a velocità differenti)**: i dispositivi vengono organizzati per classi di velocità , ognuna delle quali gestisce una classe di dispositivi con stessa velocità . Quindi per adattatori di bus si intende dei coprocessori attaccati ai bus , i quali si dividono in due : north e south : i primi a velocità maggiori , i secondi a velocità minori . In dettaglio :

